

Relatório Técnico — Trabalho Integrador

Sistema de Controle de Estoque e Pedidos — Aplicação Web (API REST + Interface)

Universidade Federal da Fronteira Sul — Ciência da Computação

GEX613 — Programação II

Autor: **Eduardo Klassen** — Matrícula 20240017565

Data: julho de 2026

1. Introdução

Este relatório descreve o desenvolvimento do backend de um sistema de controle de estoque e pedidos, proposto como Trabalho Integrador da disciplina GEX613 — Programação II. O trabalho articula conhecimentos de três componentes curriculares: Programação II (construção da aplicação), Banco de Dados (modelagem relacional) e Curso pessoal Club Full-Stack que ensinou sobre básico de arquitetura e organização do código, principalmente em PHP em vez do uso de JavaScript que foi colocado em aula.

O objetivo foi construir uma aplicação web completa: uma API REST que oferecesse operações CRUD sobre as entidades do domínio, autenticação e autorização de usuários e a lógica de negócio de uma pequena empresa que adquire materiais, comercializa produtos e controla suas movimentações financeiras; e uma interface web que consome essa API, permitindo operar o sistema pelo navegador.

O relatório está organizado em torno das duas frentes. As seções iniciais tratam do backend (arquitetura, modelagem e regras de negócio) e as seções finais descrevem o front-end (estrutura, consumo da API e responsividade), refletindo a ordem em que o trabalho foi construído.

O projeto foi desenvolvido individualmente.

2. Descrição do sistema

O sistema modela o funcionamento de uma empresa de pequeno porte sob a ótica de três fluxos integrados:

- Fluxo comercial:** clientes realizam pedidos, compostos por itens (produtos), com controle de quantidade e valores.
- Fluxo de estoque:** a empresa adquire materiais de fornecedores por meio de compras; produtos são compostos por materiais.
- Fluxo financeiro:** pedidos geram receitas e compras geram despesas, ambas refletidas em um caixa.

Esses fluxos se conectam: ao registrar uma compra, o estoque dos materiais aumenta; ao criar um pedido, o estoque dos produtos diminui. As movimentações financeiras acompanham essas operações.

3. Tecnologias utilizadas e justificativas

A escolha das tecnologias priorizou o aprendizado dos fundamentos e foi totalmente pessoal, pelo gosto por PHP.

PHP 8 com orientação a objetos, sem framework. A decisão de não adotar um framework (como Laravel) foi deliberada. Frameworks resolvem muitos problemas automaticamente — roteamento, ORM, autenticação — mas escondem como essas peças funcionam. Para esse trabalho, seguindo o curso de PHP Club Full-Stack, tentei construir manualmente o roteador, o middleware de autenticação e o acesso ao banco, o que proporciona um entendimento mais profundo.

MySQL/MariaDB como banco relacional. O domínio possui entidades fortemente relacionadas (pedidos pertencem a clientes, itens pertencem a pedidos, etc.), o que torna o modelo relacional adequado. O uso de chaves estrangeiras e constraints permite que o próprio banco garanta a integridade dos dados.

Autenticação por JWT (firebase/php-jwt). Optou-se por tokens JWT em vez de sessões tradicionais por serem *stateless*: o servidor não precisa armazenar estado de sessão, pois o próprio token carrega as informações do usuário de forma assinada. Essa abordagem é adequada a uma API REST e facilita o consumo futuro por um front-end separado.

vlucas/phpdotenv para configuração. Mantém credenciais sensíveis (senha do banco, chave secreta do JWT) fora do código-fonte, em um arquivo `.env` que não é versionado.

4. Arquitetura da aplicação

A aplicação adota uma arquitetura em camadas, com separação clara de responsabilidades — princípio central da Engenharia de Software, ensinado por curso pessoal que aborda o básico de arquitetura para funcionamento e boas práticas.

O ponto de entrada é único (`public/index.php`): toda requisição passa por ele. Em seguida, o roteador (`app/Core/Router`) identifica qual controlador deve tratar a URL e o método HTTP. Antes de chegar ao controlador, requisições protegidas passam por *middlewares* que validam o token de autenticação e o perfil do usuário. O controlador valida os dados de entrada e delega o acesso ao banco ao *model* correspondente, que executa as consultas via PDO. A resposta é sempre padronizada em JSON por um *helper* dedicado.

Essa organização reflete o padrão de separação entre **roteamento, controle, modelo e apresentação**, tornando o código previsível: cada tipo de tarefa tem um lugar definido.

5. Modelagem do banco de dados

O modelo evoluiu de uma versão inicial para uma versão revisada, resultado de um processo de análise crítica. O modelo entidade-relacionamento foi **validado em consultoria com um profissional experiente em modelagem de dados**, cuja avaliação confirmou a coerência geral das cardinalidades e dos relacionamentos, e sugeriu melhorias pontuais que foram incorporadas. As principais decisões:

Usuário com perfil por atributo, não por entidade. Em vez de criar uma entidade separada para perfis de acesso, o tipo de usuário (`admin / comum`) é um atributo da própria tabela `usuario`. Como há apenas dois perfis fixos, uma entidade separada adicionaria complexidade sem benefício.

Ligação direta entre Usuário e Pedido (rastreabilidade). Originalmente, o pedido se ligava apenas ao cliente. Identificou-se que era importante registrar *qual usuário do sistema* lançou cada pedido, e não apenas a qual cliente ele pertence. Adicionou-se, portanto, uma chave estrangeira `id_usuario` em `pedido`, melhorando a rastreabilidade das operações.

Histórico de status do pedido. Criou-se a entidade `historico_status` para registrar cada transição de status de um pedido (por exemplo, de "aberto" para "pago"), guardando o status anterior, o novo e a data da mudança. Isso permite reconstruir toda a linha do tempo de um pedido — funcionalidade semelhante ao rastreamento de uma encomenda. A cada mudança de status, uma nova linha é inserida automaticamente.

Endereço como entidade própria. Em vez de tratar endereço como atributo multivalorado de cliente e fornecedor, modelou-se como uma entidade

independente. Isso evita a colisão de identificadores que ocorreria caso se usasse um atributo multivalorado compartilhado, e permite que tanto clientes quanto fornecedores tenham endereços de forma consistente.

Relacionamento Fornecedor–Material via Compra. Avaliou-se inicialmente uma ligação direta (muitos-para-muitos) entre fornecedor e material. Concluiu-se que essa ligação seria redundante, pois o fornecedor de um material já é determinado pela compra na qual o material entrou. Removeu-se, portanto, o relacionamento direto, deixando o fornecedor associado à compra — uma decisão que elimina duplicação de informação e possibilidade de inconsistência.

Tabelas associativas para relacionamentos N:N. Dois relacionamentos muitos-para-muitos foram resolvidos com tabelas associativas: `compra_material` (uma compra contém vários materiais, cada um com quantidade e custo) e `produto_material` (um produto utiliza vários materiais em sua composição).

O modelo final possui **15 tabelas**, organizadas nos módulos de acesso, comercial, estoque, composição e financeiro.

6. Funcionalidades implementadas

- **Autenticação e autorização:** login que retorna um token JWT; verificação do token em rotas protegidas; restrição de operações de cadastro ao perfil administrador.
- **Operações CRUD** sobre todas as entidades principais.
- **Gestão de usuários:** além do login, o administrador pode listar, criar, editar e remover contas de usuário. Essa funcionalidade recebeu cuidados específicos de segurança e integridade: a senha nunca é retornada pela API (as consultas de listagem selecionam apenas as colunas públicas) e três regras impedem que o sistema fique inoperante — não é possível excluir o último administrador, excluir a própria conta durante a sessão, nem rebaixar o último administrador a usuário comum.
- **Controle de estoque automático:** ao registrar uma compra, as quantidades são somadas ao estoque dos materiais; ao criar um pedido, são subtraídas do estoque dos produtos.
- **Histórico de status:** registro automático de cada mudança de status de pedido.
- **Uso de transações:** a criação de pedidos e compras — que envolve gravar a operação e seus itens, além de atualizar o estoque — é feita dentro de uma transação de banco de dados. Assim, ou todas as operações são efetivadas, ou nenhuma é, garantindo a consistência dos dados em caso de falha.
- **Tratamento de erros orientado ao usuário:** quando uma exclusão é impedida pela integridade referencial do banco — por exemplo, ao tentar remover um fornecedor que possui compras registradas — a API não devolve um erro genérico de servidor. A violação de chave estrangeira é interceptada e convertida em uma resposta de conflito (409) com uma mensagem explicativa, informando o motivo real do bloqueio em vez de expor uma falha técnica.
- **Dashboard:** endpoint que retorna contagens e um resumo dos dados.

7. O front-end

Concluída a API, a segunda frente do trabalho foi a interface web que a consome. Ela foi construída com **HTML5, CSS3 e JavaScript puro (sem framework)**, pela mesma razão que motivou a escolha no backend: exercitar os fundamentos — manipulação do DOM, requisições `fetch` e organização do código — sem a abstração de uma biblioteca.

Organização do código. O front segue a mesma filosofia de separação de responsabilidades do backend. O CSS é dividido por componentes (tabela, modal,

barra lateral, formulário) e por página, seguindo uma convenção de nomenclatura consistente (BEM). O JavaScript é modular: cada tela tem seu próprio módulo, e funções de uso comum — requisições HTTP, autenticação, montagem do layout, componentes de modal — ficam em arquivos reutilizáveis. Todo o estilo e a lógica estão em arquivos externos; não há CSS nem JavaScript embutido no HTML.

Consumo da API. Um módulo central concentra as chamadas `fetch` à API e padroniza o tratamento das respostas. Ele anexa automaticamente o token de autenticação ao cabeçalho de cada requisição protegida e interpreta os erros devolvidos pelo servidor, traduzindo-os em mensagens claras na interface — por exemplo, exibindo o aviso de conflito quando uma exclusão é barrada por vínculo, ou redirecionando ao login quando a sessão expira.

Autenticação no cliente. Após o login, o token é guardado no navegador e reutilizado nas requisições seguintes. Enquanto o token é válido, o usuário é levado diretamente ao painel; quando expira, a aplicação o conduz de volta à tela de login. As telas internas verificam a presença de sessão antes de carregar, evitando o acesso a páginas restritas sem autenticação.

Visualização, busca e ordenação. As listagens oferecem busca textual e filtros por categoria (por exemplo, produtos ativos ou inativos). Nas telas de produtos e pedidos, os cabeçalhos das colunas permitem ordenar os registros — clicando, ordena-se de forma crescente; clicando novamente, inverte-se a ordem — atendendo ao requisito de ordenação por múltiplos campos.

Responsividade. A interface adapta-se a celular, tablet e desktop por meio de *media queries*. Em telas menores, a barra lateral de navegação recolhe-se e dá lugar a um botão de menu que a exibe como um painel deslizante; a barra de ferramentas empilha seus controles; e as tabelas passam a rolar horizontalmente, preservando a legibilidade sem quebrar o layout. No desktop, o layout original de barra lateral fixa é mantido.

8. Desafios encontrados

Esta foi a parte mais trabalhosa do projeto e também a que trouxe mais aprendizado. Os principais problemas são descritos a seguir.

O ambiente PHP no início. Antes mesmo de escrever a lógica, houve dificuldade na configuração. O `composer install` acusou a falta da extensão `zip`, e a primeira tentativa de login falhou por ausência da `pdo_mysql`. Ambos os casos foram resolvidos habilitando as extensões no `php.ini`, mas identificar a causa levou tempo. Houve ainda um alerta de segurança em uma versão da biblioteca de JWT, o que exigiu atualizar a dependência. Foram obstáculos no começo, mas que ajudaram a entender melhor como o PHP e o Composer funcionam.

API e o front-end. Este foi o desafio central do trabalho. O backend foi construído seguindo o cronograma de um curso de PHP, de forma relativamente genérica. Ao iniciar o desenvolvimento do front-end e conectar as telas à API, ficou claro que as duas partes não se encaixavam tão bem quanto o esperado. Em várias telas, a rota necessária não existia, existia com outro nome, ou retornava os dados em um formato diferente do que a interface esperava.

O caso mais evidente foi a tela de **usuários**: o front já estava pronto para listar, criar, editar e excluir usuários, mas o backend possuía apenas o registro básico, sem as rotas de CRUD. Foi necessário criar o controller e as rotas do zero para que a tela funcionasse. Situações semelhantes ocorreram em outras telas, e cada uma exigia retornar ao backend, criar ou ajustar o endpoint, e só então concluir a integração.

Conflito de nomes entre código e banco. Um problema recorrente foi o código referenciar tabelas ou colunas com um nome, enquanto o banco usava outro. As telas de pedidos e compras, por exemplo, apresentavam o erro de "tabela não

existe" porque o código buscava `historico_pedido` e `item_compra`, ao passo que o banco definia `historico_status` e `compra_material`. Como o banco já estava correto e populado, optou-se por ajustar o código para corresponder aos nomes do banco, em vez de refazer as tabelas. Identificar essas divergências exigiu testar cada tela e conferir os dados diretamente no phpMyAdmin.

Tratamento de erros e segurança. À medida que o sistema crescia, surgiram falhas ligadas à experiência de uso. Ao tentar excluir um fornecedor com compras associadas, por exemplo, o sistema exibia um "erro interno do servidor" genérico, sem informação útil ao usuário. Esses casos foram tratados em todos os pontos de exclusão, convertendo o erro técnico em uma mensagem clara. Também foi necessário reforçar a segurança: proteção do login contra tentativas repetidas, garantia de que a senha nunca é retornada pela API e regras para impedir que o sistema fique sem administrador. Nenhum desses pontos estava previsto no planejamento inicial; foram necessidades percebidas conforme o sistema passou a ser utilizado.

Ordem das migrations e detalhes do terminal. Problemas menores, porém relevantes: as tabelas precisavam ser criadas em uma ordem específica em razão das chaves estrangeiras (a de endereços depende da de fornecedores, por exemplo), e o PowerShell não aceitava o operador `<` para importar arquivos SQL, exigindo adaptar os comandos. Ambos foram documentados no guia de execução.

O principal aprendizado não foi uma tecnologia específica, mas a compreensão de que integrar o front-end e o backend é uma etapa de trabalho em si mesma. O código de cada lado pode estar correto isoladamente e ainda assim não funcionar em conjunto, e boa parte do esforço final concentrou-se nesse alinhamento, que foi subestimado no princípio.

9. Testes

A API foi testada com a ferramenta **Insomnia**, por meio de uma coleção de requisições organizada por módulo. O processo de teste validou o ciclo completo: autenticação (obtenção e uso do token), operações CRUD, a lógica de estoque (verificando que o estoque aumenta após uma compra e diminui após um pedido) e o registro do histórico de status (verificando que cada mudança gera uma nova linha no histórico).

Com o front-end integrado, os testes passaram a ser feitos também pela própria interface, percorrendo cada tela e confirmando o resultado diretamente no banco de dados (por consultas no phpMyAdmin). Esse cruzamento — operar pela tela e conferir o dado gravado — foi o que permitiu identificar e corrigir divergências entre o que o código esperava e a estrutura real do banco, como diferenças em nomes de tabelas e colunas. A responsividade foi verificada nas ferramentas de desenvolvedor do navegador, simulando as dimensões de celular, tablet e desktop.

10. Uso de ferramentas de inteligência artificial

Em conformidade com a orientação da disciplina, declara-se o uso de uma ferramenta de inteligência artificial (assistente Claude, da Anthropic) como apoio durante o desenvolvimento. Como estava sozinho e havia falta de conversas em equipe para testes e práticas, o uso foi necessário; deu-se de forma supervisionada, com todas as decisões de projeto, validações e testes conduzidos pelo autor.

- **Geração de código seguindo o padrão estabelecido:** a ferramenta auxiliou na escrita dos *models*, *controllers* e *migrations* das novas entidades (materiais, compras, receitas, despesas, endereços, caixa e histórico de status), mantendo a coerência com o código existente. No front-end, apoiou de forma análoga a estruturação dos módulos de tela, do consumo da API e das regras de estilo responsivo.
- **Revisão e correção:** verificação de sintaxe, ajuste e organização das rotas

da API e atualização da coleção de testes do Insomnia para refletir as novas entidades. Durante a integração entre front e back, o apoio foi importante para diagnosticar divergências entre o código e a estrutura real do banco — nomes de tabelas e colunas que não correspondiam — a partir da leitura dos erros retornados, com as correções sempre validadas por testes conduzidos pelo autor.

- **Implementação do front-end:** apoio na construção da interface que consome a API (telas de CRUD, autenticação no cliente, busca, ordenação e filtros) e na adaptação responsiva para dispositivos móveis, tablets e desktop.
- **Apoio na modelagem:** discussão das decisões do modelo entidade-relacionamento (a ligação direta entre usuário e pedido e o histórico de status), que foram posteriormente validadas em consultoria com um profissional de modelagem e decididas pelo autor.
- **Explicação de conceitos e documentação:** esclarecimento de conceitos técnicos e apoio na redação da documentação do projeto de modo mais detalhado para o leitor.

A ferramenta atuou como recurso de apoio, de forma análoga à consulta a documentação técnica e à consultoria com especialista.

11. Considerações finais

O desenvolvimento da aplicação permitiu aplicar, de forma integrada, conceitos de programação orientada a objetos, modelagem de banco de dados relacional, construção de interfaces web e princípios de arquitetura de software. A opção por construir tanto o backend quanto o front-end sem frameworks, embora mais trabalhosa, proporcionou compreensão aprofundada de mecanismos que normalmente são abstraídos — no servidor, o roteamento, o middleware de autenticação e o controle transacional; no cliente, o consumo de uma API por requisições assíncronas, o gerenciamento do token de sessão e a adaptação responsiva do layout.

A etapa de integração entre as duas frentes foi, em si, uma parte importante do aprendizado. Fazer o front e o back conversarem exigiu confrontar o que o código presumia com o que o banco de dados realmente continha, e boa parte do esforço final concentrou-se em alinhar essas duas visões e em tratar os erros de maneira clara para o usuário — um trabalho menos visível que a construção das telas, mas essencial para que o sistema funcionasse de forma coesa.

Repositório

Código-fonte: <https://github.com/Eduardoklassen/Projeto-Integrador-UFFS>